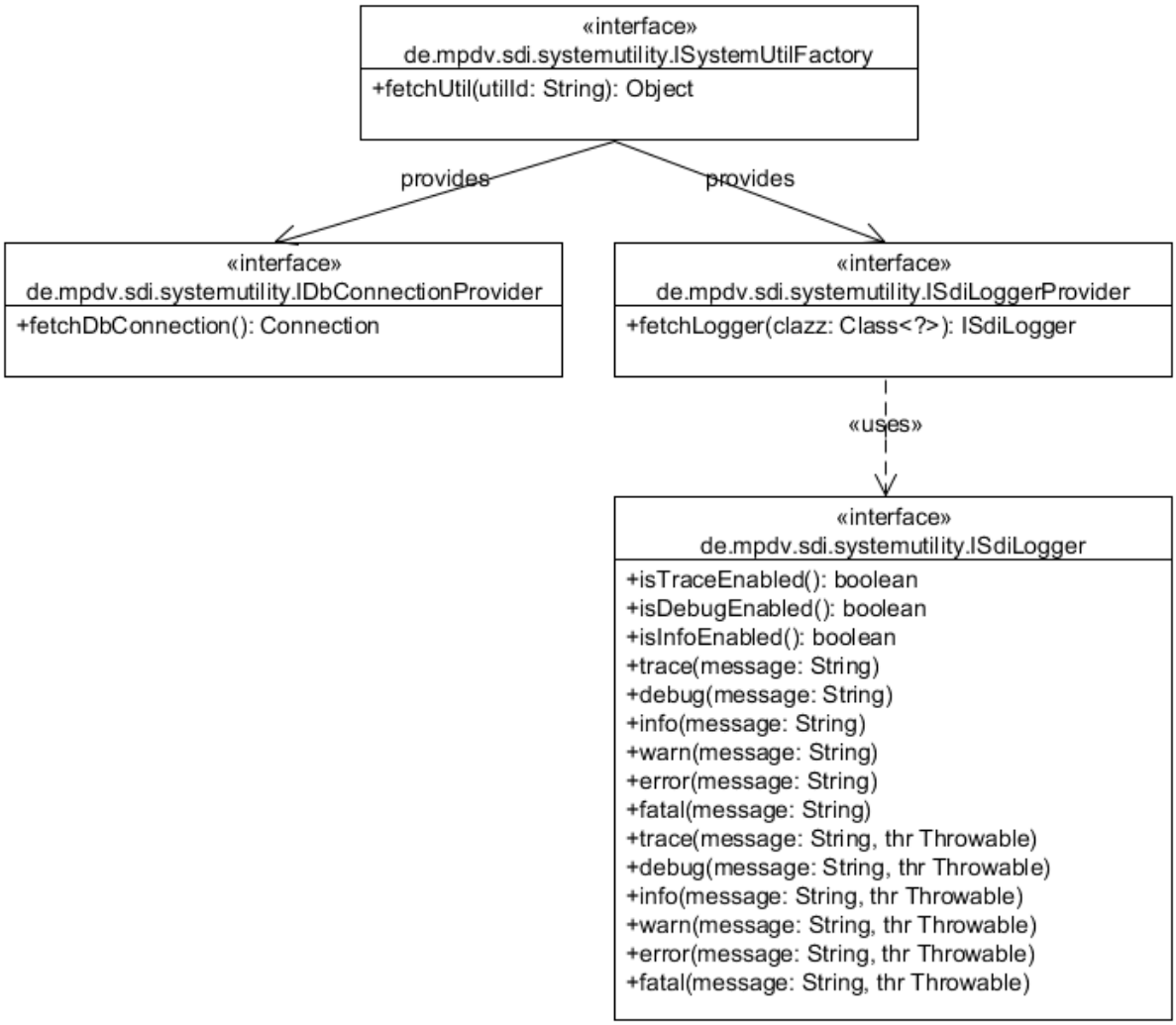
1 System Utitilies

1.1 Introduction

The System Utilities are a pool of system functions. For the application developers, these system functions are available as part of the Simple Developer Interface (SDI).

1.2 Schnittstellenbeschreibung



1.3 Interface ISystemUtilFactory

You use this interface to get access to the system utility functions, such as logging.

You can find a list of all available utilities in the following.

Method	Description
fetchUtil()	Provides an instance of the utility for the utilID. **Return type:** Object - interface for the respective utility. See the below list of utilities. **Input**: String utilId – the ID of the required System Utility. The ID is case sensitive! A list of all available IDs can be found in the utility list below. **Exception**: If "utilID“ is not known, an IllegalArgumentException is thrown.

1.3.1 Utility list

utilId	Interface of the returned utility	Available as of SPX
LoggerProvider	ISdiLoggerProvider	
DbConnectionProvider	IDbConnectionProvider	
ToNativeSqlConverter	IToNativeSqlConverter	
ToNativeSqlWithParamsConverter	IToNativeSqlWithParamsConverter	
ToNativeSqlWithInlinedParamsConverter	IToNativeSqlWithInlinedParamsConverter	
ServiceConfigProvider	IServiceConfigProvider	
HydraCaller	IHydraCaller	
PdmStringParser	IPdmStringParser	
DataTableBuilder	IDataTableBuilder	

DataTableModificator	IDataTableModificator	
FeatureSetChecker	IFeatureSetChecker	
DbTypeRetriever	IDbTypeRetriever	
UserExitProvider	IUserExitProvider (deprecated as of SP8: replaced by: IExitMethodExecutionManager)	
UserExitFromScopeProvider	IUserExitFromScopeProvider (deprecated as of SP8: replaced by: IExitMethodExecutionManager)	
ServiceCaller	IServiceCaller	
FormulaProvider	IFormulaProvider	
AcronymMappingProvider	IAcronymMappingProvider	
DataAvailabilityChecker	IDataAvailabilityChecker	
JhydradirPathProvider	IJhydradirPathProvider	
JhydradirInstancePathProvider	IJhydradirInstancePathProvider	
JhydradirTempPathProvider	IJhydradirTempPathProvider	
GlobalConfigValueProvider	IGlobalConfigValueProvider	
InstanceConfigValueProvider	IInstanceConfigValueProvider	
HydraPathReadFileAction	IHydraPathReadFileAction	
HydraPathWriteFileAction	IHydraPathWriteFileAction	
HydraPathRenameFileAction	IHydraPathRenameFileAction	
HydraPathDeleteFileAction	IHydraPathDeleteFileAction	
HydraPathFileExistsAction	IHydraPathFileExistsAction	
HydraPathCreateFolderAction	IHydraPathCreateFolderAction	

HydraPathDeleteFolderAction	IHydraPathDeleteFolderAction	
HydraPathDownloadContentAction	IHydraPathDownloadContentAction	
HydraPathUploadContentAction	IHydraPathUploadContentAction	
HydraPathListContentAction	IHydraPathListContentAction	
HydraPathIsDirectoryAction	IHydraPathIsDirectoryAction	
PersonCardIdChecker	IPersonCardIdChecker	
ResponsibilityAreaChecker	IResponsibilityAreaChecker	
PersonResponsibilityAreaChecker	IPersonResponsibilityAreaChecker	
SqlGenerator	ISqlGenerator	SP7
MemoryChecker	IMemoryChecker	SP7
MandatorySpecialParameterValidator	IMandatorySpecialParameterValidator	SP7
ExitMethodExecutionManager	IExitMethodExecutionManager	SP8
SdiDataRowCopyUtil	ISdiDataRowCopyUtil	SP9
DataTableToStreamConverter	IDataTableToStreamConverter	SP9
StreamToDataTableConverter	IStreamToDataTableConverter	SP9
ResultTransformationManager	IResultTransformationManager	SP9

1.4 Interface IDbConnectionProvider

This interface provides database connections with connection objects.

1.5 Interface ISdiLoggerProvider

You use this interface to create loggers for a specific class. You can then address these loggers separately in the logging configuration.

1.6 Interface ISdiLogger

You can use the interface ISdiLogger to log outputs on different priority levels. The following ascending order applies for the priority: trace, debug, info, warn, error, fatal.

1.7 Interface IToNativeSqlConverter

You can use the interface IToNativeSqlConverter to convert MPDV-SQL (SQL with MPDV extensions, e.g. $lang or individual escapes) into native SQL without bind variables.

Method	**Description**
toNativeSql()	Converts MPDV-SQL into native SQL **Return type:** String - native SQL **Input**: String sql – MPDV SQL **Exception**: In case of an error, an SdiException including a language key and (optional) parameters is delivered.

1.8 Interface IToNativeSqlWithParamsConverter

You can use the interface IToNativeSqlWithParamsConverter to convert MPDV-SQL (SQL with MPDV extensions, e.g. $lang or individual escapes) into native SQL with bind variables.

Method	**Description**
toNativeSqlWithParams()	Converts MPDV-SQL into native SQL with bind variables. **Return type:** NativeSqlData – for a description, refer to the description of the data class

	Input: String sql – MPDV SQL Map<String, MpdvSqlParam> parameterMap – map including bind variables (name of variable value) – for the description of MpdvSqlParam, refer to the data class **Exception**: In case of an error, an SdiException including a language key and (optional) parameters is delivered.

1.8.1 Data class MpdvSqlParam

Variable with type	**Description**
String paramName	Name of the bind variable (placeholder from statement without leading :)
Object Value	Value of the bind variable
int sqlType	SQL type matching the value type of the bind variable (see java.sql.Types)

1.8.2 Data class NativeSqlParam

Variable with type	**Description**
String paramName	Name of the bind variable (placeholder from statement without leading :)
Object	Value of the bind variable

Value	
int sqlType	SQL type matching the value type of the bind variable (see java.sql.Types)

1.8.3 Data class NativeSqlData

Variable with type	**Description**
String nativeSql	Native SQL statement - placeholders for bind variables are replaced by ? (as required for java.sql.PreparedStatement)
List<NativeSqlParam> nativeSqlParamList	The values of bind variables (in the correct order)

1.9 Interface IToNativeSqlWithInlinedParamsConverter

You can use the interface IToNativeSqlWithInlinedParamsConverter to convert MPDV-SQL (SQL with MPDV extensions, e.g. $lang or individual escapes) into native SQL with bind variables. The statement returned can be used by any database tool using JDBC.

Method	**Description**
toNativeSqlWithInlinedParams ()	Converts MPDV-SQL with bind variables into native SQL including bind variables **Return type:** String - native SQL including bind variables **Input**: String sql – MPDV SQL

	Map<String, MpdvSqlParam> parameterMap – map including bind variables (name of variable value) – for the description of MpdvSqlParam, refer to the data class (previous section) **Exception**: In case of an error, an SdiException including a language key and (optional) parameters is delivered.

1.10 Interface IServiceConfigProvider

You can use the interface IServiceConfigProvider to import the XML configuration for a service (from the listInterpreter folder). Scopes are taken into account. Using the returned object, you can perform queries via xpath for the configuration.

Method	**Description**
fetchServiceConfig()	Loads the service configuration for the service and supplies an object to access the configuration via xpath. **Return type:** IServiceConfig – Object to access configuration via xpath. See description in the next section. **Input**: String serviceName – name of the service **Exception**: In case of an error, an SdiException including a language key and (optional) parameters is delivered.

1.11 Interface IServiceConfig

The interface IServiceConfig includes the loaded configuration for a service. You can use the interface to query/access the configuration via xpath.
For further details on the xpath syntax, see e.g. http://www.w3schools.com/xpath/

Method	Description
getStringValues()	Applies xpath to the configuration and returns the string values of detected objects. Supported objects (and return values) are: - Elements (tag name) - Attributes (value) - Text (text) - Comments (text) **Return type:** List<String> - string values of the identified objects **Input**: String xPath – xPath expression **Exception**: In case of an error, an SdiException including a language key and (optional) parameters is delivered.

1.12 Interface IHydraCaller

You can use the interface IHydraCaller to build a PDM string and to call a PDM dialog (DLG=…). The results of the call are returned.

Method	Description
addString()	Adds a string value for an acronym to the PDM string.

	Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). **Return type:** . **Input**: String acronym – the acronym String value - the value **Exception**: -
addInteger()	Adds an integer value for an acronym to the PDM string Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). **Return type:** . **Input**: String acronym – the acronym Integer value - the value **Exception**: -
addDecimal()	Adds a BigDecimal value for an acronym to the PDM string

	Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). **Return type:** . **Input**: String acronym – the acronym BigDecimal value – the value **Exception**: -
addBoolean()	Adds a Boolean value for an acronym to the PDM string (J for true and N for false) Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). **Return type:** . **Input**: String acronym – the acronym Boolean value - the value **Exception**: -
addDate()	Adds a date value for an acronym to the PDM string (format = MM/DD/YYYY)

	Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). **Return type:** . **Input**: String acronym – the acronym Calendar value – the value **Exception**: -
addTime()	Adds a time value for an acronym to the PDM string (in seconds since 00:00:00) Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). **Return type:** . **Input**: String acronym – the acronym Calendar value – the value **Exception**: -
addTimestamp()	Adds a time stamp value for an acronym to the PDM string (format = MM/DD/YYYY HH:mm:ss)

	Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). **Return type:** . **Input**: String acronym – the acronym Calendar value – the value **Exception**: -
addPdmString()	Adds a (sub-)PDM string (e.g. ACR1=VAL1|ACR2=VAL2|). Note: This method parses the string into its elements and combines the elements with the other values from other methods later on. Important: If the value is empty, the complete acronym is not passed to the PDM service. If this behavior is not wanted, use the method *NoSkipEmptyValue(). **Return type:** . **Input**: String pdmString – the (sub-) PDM string **Exception**: -
addStringNoSkipEmptyValue()	Adds a string value for an acronym to the PDM string.

	Return type: . **Input**: String acronym – the acronym String value - the value **Exception**: -
addIntegerNoSkipEmptyValue()	Adds an integer value for an acronym to the PDM string **Return type:** . **Input**: String acronym – the acronym Integer value - the value **Exception**: -
addDecimalNoSkipEmptyValue()	Adds a BigDecimal value for an acronym to the PDM string **Return type:** . **Input**: String acronym – the acronym BigDecimal value – the value **Exception**:

	-
addBooleanNoSkipEmptyValue()	Adds a Boolean value for an acronym to the PDM string (J for true and N for false) **Return type:** . **Input**: String acronym – the acronym Boolean value - the value **Exception**: -
addDateNoSkipEmptyValue()	Adds a date value for an acronym to the PDM string (format = MM/DD/YYYY) **Return type:** . **Input**: String acronym – the acronym Calendar value – the value **Exception**: -
addTimeNoSkipEmptyValue()	Adds a time value for an acronym to the PDM string (in seconds since 00:00:00) **Return type:** . **Input**: String acronym – the acronym

	Calendar value – the value **Exception**: -
addTimestampNoSkipEmptyValue()	Adds a time stamp value for an acronym to the PDM string (format = MM/DD/YYYY HH:mm:ss) **Return type:** . **Input**: String acronym – the acronym Calendar value – the value **Exception**: -
addPdmStringNoSkipEmptyValue()	Adds a (sub-)PDM string (e.g. ACR1=VAL1|ACR2=VAL2|). Note: This method parses the string into its elements and combines the elements with the other values from other methods later on. **Return type:** . **Input**: String pdmString – the (sub-) PDM string **Exception**: -
callHydra()	Executes the PDM call and provides the results. In case of an error, the HydraReturnValue has a return code != 0

	Return type: HydraCallResult – description, see below **Input**: - **Exception**: -

1.12.1 Data class HydraCallResult

Variable with type	Description
IDataTable resultTable	The payload of the return string / the file of the PDM call All fields of the string type For further details on the type, refer to the data types for "Simple External Services“
HydraReturnValue returnValue	Result of calling (RET/KT/LT/DATEI)

1.12.2 Data class HydraReturnValue

Variable with type	Description
Integer returnCode	Value of the result string of the RET acronym. In case of an error !=0.
String shortText	Value of the result string of the KT acronym
String	Value of the result string of the LT acronym

longText	
String fileName	Value of the result string of the DATEI acronym

1.13 Interface IPdmStringParser

You can use the interface IPdmStringParser to parse PDM strings. Supported methods are the key/value method and the value method.

Method	**Description**
parseKeyValueString()	Parses a PDM string in Key/Value format (e.g. ACR1=Wert1|ACR2=Wert2|). Escaping reserved characters (e.g. |) is supported. **Return type:** Map<String, String> - acronyms including their values **Input**: String input – PDM string **Exception**: -
parseUnescapedKeyValueString()	Parses a PDM string in Key/Value format (e.g. ACR1=Wert1|ACR2=Wert2|). Escaping reserved characters (e.g. |) is NOT supported. **Return type:** Map<String, String> - acronyms including their values

	Input: String input – PDM string **Exception**: -
parseValueOnlyString()	Parses a PDM string including only values (e.g. Wert1|Wert2|) Escaping reserved characters (e.g. |) is supported. **Return type:** List<String> - the values **Input**: String input – PDM string **Exception**: -
parseUnescapedValueOnlyString()	Parses a PDM string including only values (e.g. Wert1|Wert2|) Escaping reserved characters (e.g. |) is NOT supported. **Return type:** List<String> - the values **Input**: String input – PDM string **Exception**: -

1.14 Interface IDataTableBuilder

Use the interface IDataTableBuilder to create DataTables.

Method	Description
setDataTableId()	Sets table ID **Return type:** IDataTableBuilder – reference to the builder **Input**: String dataTableId – ID of the DataTable **Exception**: -
addCol()	Adds a column to the table - must not be requested if rows/data have already been added. For further details on the DataType type, refer to the data types of "Simple External Services“ **Return type:** IDataTableBuilder – reference to the builder **Input**: String colName – column name DataType columnType – column type **Exception**: IllegalStateException – if data / a row has been added
addRow()	Adds an empty row to the table **Return type:** IDataTableBuilder – reference to the builder

	Input: - **Exception**: -
addRow()	Adds a row including values to the table **Return type:** IDataTableBuilder – reference to the builder **Input**: Object… values – the values (there must be as many values as columns) **Exception**: IllegalArgumentException – if the number of values does not match the number of columns
value()	Sets the value in the next column of the current row **Return type:** IDataTableBuilder – reference to the builder **Input**: Object value – the value **Exception**: IllegalStateException – if no row is generated IndexOutOfBoundsException – if the last column is reached and an attempt is made to set a value
build()	Generate DataTable For further details on the type IDataTable, refer to the data types of "Simple External Services“

	Return type: IDataTable – the table **Input**: - **Exception**: -

1.15 Interface IDataTableModificator

You use the interface IDataTableModificator to modify DataTables. A copy is made prior to processing. Consequently, changes are not made to the original DataTable.

Method	**Description**
setDataTable()	Setting the DataTable **Return type:** IDataTableModificator – reference to the modificator **Input**: IDataTable table – the DataTable **Exception**: -
addRow()	Adds an empty row to the table **Return type:** IDataTableModificator – reference to the modificator **Input**:

	- **Exception**: -
removeRow()	Removes a row from the table **Return type:** IDataTableModificator – reference to the modificator **Input**: int rowIndex – index of the row **Exception**: IndexOutOfBoundsException – if the row with the index does not exist
addColumn()	Adds a column to the table For further details on the DataType type, refer to the data types of "Simple External Services“ **Return type:** IDataTableModificator – reference to the modificator **Input**: String columnName – column name DataType columnType – column type **Exception**: IllegalStateException – if a column with the name exists
removeColumn()	Removes a column from the table **Return type:**

	IDataTableModificator – reference to the modificator **Input**: String columnName – column name **Exception**: IllegalStateException – if a column with the name does not exist
setDataTableId()	Sets table ID **Return type:** IDataTableModificator – reference to the modificator **Input**: String dataTableId – ID of the DataTable **Exception**: -
setValue()	Sets the value in a cell **Return type:** IDataTableModificator – reference to the modificator **Input**: int rowIndex – index of the row String columnName – name of column Object value – the value **Exception**: IndexOutOfBoundsException – if the row with the index does not exist

	IllegalStateException – if a column with the name does not exist
getModifiedTable()	Return of the modified DataTable **Return type:** IDataTable – the table **Input**: - **Exception**: -

1.16 Interface IFeatureSetChecker

You can use the interface IFeatureSetChecker to check if a feature set is active.

Method	**Description**
isFeatureSetActive()	Checks if feature set is active **Return type:** Boolean - true if active, otherwise false **Input**: String name – name of feature set **Exception**: SdiException with key lkErrorCheckingFeatureSet if an error occurred during the check

1.17 Interface IDbTypeRetriever

You can use this interface to identify the database type (Oracle or MS SQL Server)

Method	Description
getDbType()	Returns the DB type as string Oracle = "ORACLE“ MS SQL Server = "SQL_SERVER“ **Return type:** String – the DB type **Input**: - **Exception**: -
isOracleDb()	Verifies if it is an Oracle DB **Return type:** Boolean – true if it is an Oracle DB **Input**: - **Exception**: -
isSqlServerDb()	Verifies if it is an MS SQL Server DB **Return type:** Boolean – true if it is an MS SQL Server DB **Input**: - **Exception**: -

1.18 Interface IUserExitProvider

Deprecated as of SP8: can be replaced by IExitMethodExecutionManager.

You can use this interface to generate a user exit object from a user exit class (scope is identified automatically).

Method	**Description**
fetchUserExit()	Generates a user exit object from the user exit class of the current service. The class is loaded from the scope with the highest priority. **Return type:** IUserExit – the user exit object **Input**: - **Exception**: -
fetchUserExit()	Generates a user exit object from the specified user exit class. The class is loaded from the scope with the highest priority. **Return type:** IUserExit – the user exit object **Input**: String userExitId – the full class name of the user exit (including package) **Exception**: -

1.19 Interface IUserExitFromScopeProvider

Deprecated as of SP8: can be replaced by IExitMethodExecutionManager.

You can use this interface to generate a user exit object from a user exit class (scope is specified).

Method	Description
fetchUserExitFromScope()	Generates a user exit object from the user exit class of the current service. The class is loaded from the specified scope. **Return type:** IUserExit – the user exit object **Input**: String scope – the scope (LOCAL, CUSTOM, STANDARD) **Exception**: -
fetchUserExitFromScope()	Generates a user exit object from the specified user exit class. The class is loaded from the specified scope. **Return type:** IUserExit – the user exit object **Input**: String userExitId – the full class name of the user exit (including package) String scope – the scope (LOCAL, CUSTOM, STANDARD)

	Exception: -

1.20 Interface IUserExit

You can use this interface to access a loaded user exit class.

Method	Description
getScope()	Identifies the scope that was used to load the user exit class. **Return type:** String – the scope (LOCAL, CUSTOM, STANDARD) **Input**: - **Exception**: -
invoke()	Calls a method of the user exit class **Return type:** Object - the return object of the user exit (the class is different for each user exit) **Input**: String methodName – method name Object context – context object (the class depends on the source that called the user exit)

	Object param – parameter object (the class is different for each user exit) **Exception**: -

1.21 Interface IAcronymMappingProvider

You can use this interface to access acronym mapping.

Method	Description
getAcronymMapping()	Loads the acronym mapping for the current service **Return type:** IAcronymMapping – the mapping object **Input**: - **Exception**: -

1.21.1 Interface IAcronymMapping

You use this interface to provide access to the acronym mapping.

Method	Description
mapFromServiceAcronym()	Returns mapping for the service acronym **Return type:** String – the mapping (null if not available) **Input**:

	String serviceAcronym – the service acronym **Exception**: -
mapToServiceAcronym()	Returns the service acronym for the mapping **Return type:** String – the service acronym (null if not available) **Input**: String mappedAcronym - the mapped acronym **Exception**: -
getFromServiceAcronymMapping ()	Provides a map with assignment service acronym -> mapped acronym **Return type:** Map<String, String> - the map including assignment **Input**: - **Exception**: -
getToServiceAcronymMapping ()	Provides a map with assignment mapped acronym -> service acronym **Return type:** Map<String, String> - the map including assignment **Input**:

	- **Exception**: -

1.22 Interface IFormulaProvider

You can use this interface to create and work with formulas based on a term.

Method	Description
createFormula ()	Creates the formula object for the term **Return type:** IFormula – the formula object **Input**: String formulaTerm – the term MissingParamHandling missingParamHandling – specifies the next steps if one of the placeholders is null or not set in the formula (ERROR or IGNORE are possible). **Exception**: -

1.22.1 Interface IFormula

You use this interface to provide access to the generated formula

Method	Description
getPlaceHolders()	Returns all placeholders (variables) used in the formula **Return type:**

	List<String> – the placeholders **Input**: - **Exception**: -
setStringParam()	Sets a placeholder with string value **Return type:** - **Input**: String paramName – placeholder name String value - the value **Exception**: -
setIntegerParam()	Sets a placeholder with integer value **Return type:** - **Input**: String paramName – placeholder name Integer value - the value **Exception**: -
setDecimalParam()	Sets a placeholder with decimal value **Return type:** -

	Input: String paramName – placeholder name BigDecimal value – the value **Exception**: -
setBooleanParam()	Sets a placeholder with Boolean value **Return type:** - **Input**: String paramName – placeholder name Boolean value - the value **Exception**: -
evaluate()	Calculates the formula **Return type:** IFormulaEvaluationResult – result of the calculation (or null if value is not set - IGNORE mode) **Input**: - **Exception**: SdiException – If value is not set (ERROR mode)

1.22.2 Interface IFormulaEvaluationResult

You use this interface to provide access to the result of formula calculation.

Method	Description
getStringResult()	Gets the result as string (if possible) **Return type:** String - result **Input**: - **Exception**: -
getIntegerResult()	Gets the result as integer (if possible) **Return type:** Integer result **Input**: - **Exception**: -
getDecimalResult()	Gets the result as decimal (if possible) **Return type:** BigDecimal - result **Input**: - **Exception**: -
getBooleanResult()	Gets the result as Boolean (if possible)

	Return type: Boolean - result **Input**: - **Exception**: -

1.23 Interface IServiceCaller

You use this interface for the internal request of another service.

Method	Description
callService()	Internal service call **Return type:** IServiceCallResult – call result **Input**: String serviceName – name of service ColumnConfigurator colConf – column configurator List<SpecialParam> specialParams – Special Parameters List<FilterParam> filterParams – Filter Parameters (for the Where clause in SQL) **Exception**: SdiException – If a special parameter or filter parameter is null or if the parameter has a data type that is not supported, if the service provides several tables or additional information on the result, if the service execution results in an error.

1.23.1 Interface IServiceCallResult

You use this interface to provide access to the result of a service call.

Method	Description
getResultTable()	Provides the result table **Return type:** IDataTable – the table (or null if no table is provided) **Input**: - **Exception**: -

1.23.2 Class FilterParam

This is the data type that includes information on a filter parameter. The data of this type cannot be changed.

Method	Description
getAcronym	Provides the acronym of this filter parameter **Return type:** String
getOperator	Provides the operator of this filter parameter **Return type:** OperatorType (enum)
getValue	Provides the value of this filter parameter **Return type**: Object The actual data type depends on the parameter type. These types are possible: - String - Integer - BigDecimal - Boolean

	- Byte[] - Calendar - String[] - Integer[] - BigDecimal[] - Boolean[] - Calendar[]

1.24 Interface IDataAvailabilityChecker

You can use this interface to check data availability for product(s)/object(s) for a specific period of time.

Method	Description
checkAvailability()	Checks availability **Return type:** DataAvailability – result of checking **Input**: DataAvailabilityCheckData checkData – parameters for checking **Exception**: -

1.24.1 Class DataAvailabilityCheckData

These are the call parameters for the check

Method	Description
getEvaluationType	Type of evaluation (based on shifts or periods of time) SHIFT TIMERANGE **Return type:** String

getBegin	Start of evaluation period **Return type**: Calendar
getEnd	End of evaluation period **Return type**: Calendar
isCheckCurrentAvailability	Check if current data actually include values matching the relevant period of time **Return type:** Boolean
getCheckObjects	Combinations of product/product object **Return type:** DataAvailabilityObject[]

1.24.2 Class DataAvailabilityObject

These are the call parameters for the check

Method	**Description**
getProduct	Product: MDE ADE WRM **Return type:** String
getProductObject	Object pertaining to the product MDEPRO, EREIGMDE, RES_STATUS ADEPRO, ANR WRMPRO, EREIGWRM **Return type:** String

1.24.3 Class DataAvailability

This is the result of the availability check.

Method	Description
isNoDataAvailable	Flag that specifies if any data is available (if not, you need not select data). **Return type**: Boolean
isLongTermDataNeeded	Flag that specifies if archive tables must be included **Return type**: Boolean
getInfoLanguageKey	Additional information if data is only partially available - language key of message **Return type:** String
getInfoShortMessage	Additional information if data is only partially available - brief information (internal) of message **Return type:** String
getInfoParams	Additional information if data is only partially available - parameters for the language key **Return type**: Object[]

1.25 Interface IJhydradirPathProvider

You use this interface to identify the path to the JDIR or the JHYDRADIR directory, which includes the configuration of the WSP.

Method	Description
getPath(): String	**Return type:** String – path to JDIR

	Input:

1.26 Interface IJhydradirInstancePathProvider

This interface is used to identify the path to the instance directory of the JDIR or JHYDRADIR directory in the MOC subfolder where the instance configuration of the WSP is located.

Method	**Description**
getPath(): String	**Return type:** String – path to JDIR instance directory. **Input**:

1.27 Interface IJhydradirTempPathProvider

You use this interface to identify the path to the directory for temporary data of the JDIR or the JHYDRADIR directory.

Method	**Description**
getPath(): String	**Return type:** String – path to the temporary directory to JDIR or JHYDRADIR **Input**:

1.28 Reading settings from the configuration files

1.28.1 WSP configuration files

When the WSP is installed, there are two configuration files "config.properties" where settings are made using key/value pairs.

File	Example
Global configuration file	\\myserver\mip1\jdir\MOC\config.properties
Instance configuration file	\\myserver\mip1\jdir\MOC\1\config.properties

Example:

```
################################################
############INSTANCE CONFIGURATION (1)#############
################################################
# Please make sure that after configuration values is no TAB or SPACE

# Config reload timeout in seconds
# After this timeout it is checked, if the instance configuration (this file) has changed and the config must be reloaded
configreload.timeout=10

development.mode=1

################################################
############DATABASE CONFIGURATION###############
################################################

# The name of the instance. This is used for configuration of database connection pool
instance.name=mip1

# DB-type (valid values: oracle or sqlserver):
# Needed for the mapping of SQL errors
# db.type=oracle
db.type=sqlserver

# Is the Database a unicode DB? If yes the value must be 1 else 0
db.unicode=1
...
```

1.28.2 Interface IGlobalConfigValueProvider

The IGlobalConfigValueProvider interface allows configuration values to be read from the **global** instance configuration file.

Method	Description
getConfigValue(String key): String	**Input**: String: key for the configuration value **Return type:** String – configuration value

1.28.3 Interface IInstanceConfigValueProvider

The IInstanceConfigValueProvider interface enables configuration values to be read from **the instance** configuration file.

Method	Description
getConfigValue(String key): String	**Input**:

	String: key of the configuration value **Return type:** String – configuration value

1.29 Interface IHydraPathReadFileAction

You use this interface to read files that you can reach via configured path.

Method	Description
openFile(String pathName, String subPath): InputStream	**Return type:** InputStream – opened file stream **Input**: String pathName: name of the path configuration Paths are configured in the system and identified here by the name of the configuration. String subPath: Subpath in path or NULL, if direct path

1.30 Interface IHydraPathWriteFileAction

You use this interface to write files that you can reach via configured path.

Method	Description
openFile(String pathName, String subPath): void	**Return type:** **Input**: InputStream fileData: file data to be written String pathName: configuration name of a path.

	String subPath: Subpath in path or NULL, if direct path

1.31 Interface IHydraPathRenameFileAction

You use this interface to rename files that you can reach via configured path.

Method	Description
renameFile (String pathName, String sourceName, String targetName): boolean	**Return type:** Boolean – TRUE if renaming was successful; otherwise FALSE **Input**: String pathName: configuration name of a path String sourceName: source file name String targetName: target file name

1.32 Interface IHydraPathDeleteFileAction

You use this interface to delete files that you can reach via configured path.

Method	Description
deleteFile(String pathName, String fileName): boolean	**Return type:** Boolean – TRUE if successful; otherwise FALSE **Input**: String pathName: path name String fileName: file name including potential sub-directories

1.33 Interface IHydraPathFileExistsAction

You use this interface to check if files exist that you can reach via configured path.

Method	Description
exists(String pathName, String subPath): boolean	**Return type:** Boolean – TRUE if file exists; otherwise FALSE **Input**: String pathName: path name String subPath: subpath in the path

1.34 Interface IHydraPathCreateFolderAction

You use this interface to create directories that you can reach via configured path.

Method	Description
createFolder(String pathName, String subPath, String folderName): boolean	**Return type:** Boolean – TRUE if successful; otherwise FALSE **Input**: String pathName: path name String subPath: subpath in the path String folderName name of the directory to be created

1.35 Interface IHydraPathDeleteFolderAction

You use this interface to delete directories that you can reach via configured path.

Method	Description

deleteFolder(String pathName, String subPath, String folderName): boolean	**Return type:** Boolean – TRUE if successful; otherwise FALSE **Input**: String pathName: path name String subPath: subpath in the path String folderName name of the directory to be deleted

1.36 Interface IHydraPathDownloadContentAction

You use this interface to download complete directories that you can reach via configured path.

Method	**Description**
downloadContent(String pathName, String subPath, String localDestinationFolderPath, boolean recursive): void	**Return type:** **Input**: String pathName: path name String subPath: subpath in the path String localDestinationFolderPath name of the local target directory Boolean recursive: if TRUE all subdirectories are included, otherwise only files included in this directory

1.37 Interface IHydraPathUploadContentAction

You use this interface to upload a local directory to a directory that you can reach via configured path.

Method	**Description**

uploadContent(String pathName, String subPath, String localSourceFolder): void	**Return type:** **Input**: String pathName: path name String subPath: subpath in the path String localSourceFolder name of the local source directory

1.38 Interface IHydraPathListContentAction

You use this interface to list the content of a directory that you can reach via configured path.

Method	**Description**
uploadContent(String pathName, String subPath, boolean recursive): List<IHydraPathElement>	**Return type:** List<IHydraPathElement>: List including directory contents **Input**: String pathName: path name String subPath: subpath in the path Boolean recursive: if TRUE all subdirectories are included, otherwise only files included in this directory

1.39 Interface IHydraPathIsDirectoryAction

You use this interface to check if a path in a directory that you can reach via configured path is a directory or not.

Method	**Description**

isDirectory(String pathName, String subPath): boolean	**Return type:** boolean – TRUE if subPath is a directory; otherwise FALSE **Input**: String pathName: path name String subPath: subpath in the path

1.40 Interface IPersonCardIdChecker

You use this interface to check if a person exists, if the person is not locked and has joined/not yet left the company. The check uses the badge number.

Method	**Description**
checkCardId()	Checks the person using the badge number. The specified date is used to verify the date of joining/leaving the company. **Return type:** IPersonCardIdCheckResult – result of the check **Input**: String cardId – the badge number Calendar checkDate – date to check the date of joining/leaving the company **Exception**: SesException – if an error occurred with database access
checkCardIdForToday()	Performs the check as described for checkCardId and uses the current day as the checkDate.

	Return type: IPersonCardIdCheckResult – result of the check **Input**: String cardId – the badge number **Exception**: SesException – if an error occurred with database access

1.40.1 Interface IPersonCardIdCheckResult

This interface provides the result of the personnel verification using the badge number

Method	Description
getPersonId()	Provides the personnel number for the badge number (if person exists) **Return type:** Integer – the personnel number **Input**: - **Exception**: -
getUserId()	Provides the user assigned to the person (if person exists and a user is assigned) **Return type:** String – the user **Input**: -

	Exception: -
getCheckResult()	Provides the result of the check **Return type:** PersonCardIdCheckState – the actual result of the check **Input**: - **Exception**: -

1.40.2 Enum PersonCardIdCheckState

Includes the result of the personnel verification

Value	Description
PERSON_NOT_EXISTING	Person does not exist
PERSON_LOCKED	Person has been locked
PERSON_NOT_JOINED_YET	Person has not yet joined the company by the date of check
PERSON_ALREADY_LEAVED	Person has already left the company by the date of check
CHECK_OK	Everything all right

1.41 Enum CheckResponsibilityAreaMode

Modes to check the authorizations for responsibility areas

Value	Description

VIEW	Single authorization for viewing
USE	Single authorization for using
INSERT	Single authorization for creating
UPDATE	Single authorization for editing
DELETE	Single authorization for deleting
LOCK	Single authorization for locking
COPY	Single authorization for copying
LIST	Single authorization for listing
ANY	There is a row in the table for responsibility areas. All single authorizations may have any value.

1.42 Interface IResponsibilityAreaChecker

You can use this interface to check whether or not a user is authorized for a specific responsibility area.

Method	**Description**
isResponsibilityAreaPermissionGranted(ResponsibilityAreaCheckerParam param): boolean	**Return type:** Boolean – TRUE the user has the required authorization **Input**: ResponsibilityAreaCheckerParam param: parameter structure

1.42.1 Class ResponsibilityAreaCheckerParam

Parameter class to check responsibility areas

Field	Description
String: user	User
String: responsibilityArea	Responsibility area to be checked (if it is allowed for the user)
CheckResponsibilityAreaMode: mode	Mode of checking (optional, if left out, ANY)

1.43 Interface IPersonResponsibilityAreaChecker

You can use this interface to check whether or not a user is authorized for a specific person.

Method	Description
isPersonResponsibilityAreaPermissionGranted(PersonResponsibilityAreaCheckerParam param): PersonResponsibilityAreaCheckerResult	**Return type:** PersonResponsibilityAreaCheckerResult **Input**: PersonResponsibilityAreaCheckerParam param: parameter structure

1.43.1 Enum PersonResponsibilityAreaCheckerResult

Parameter class to check personnel responsibility areas

Value	Description
ALLOWED	Authorization available
NOT_ALLOWED	Authorization not available
PERSON_NOT_EXISTING	Person to be checked does not exist

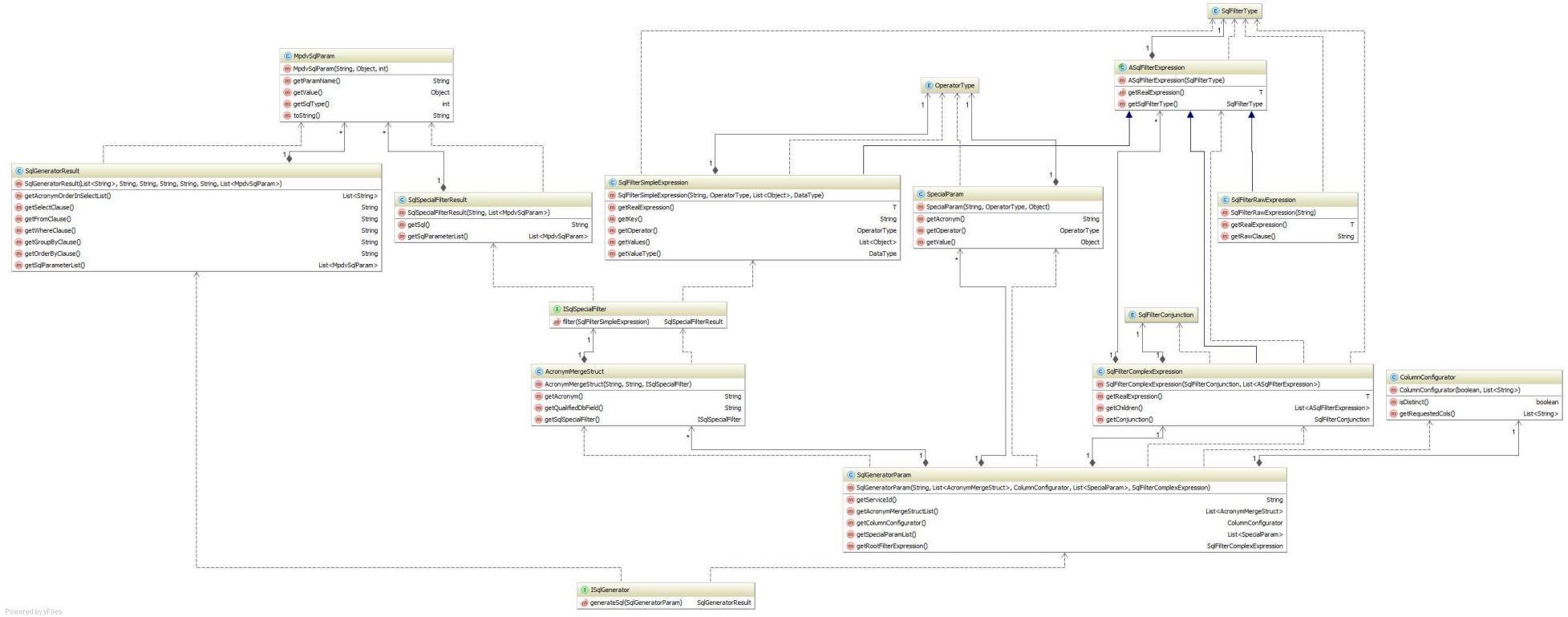
1.44 Interface ISqlGenerator

You use this interface to generate SQL statements using a service configuration. This means: Via the interface, you can generate the SQL that is executed via InterpretedJavaService2, but you do not execute it.

Note: If the acronyms in the service, which is intended to be used as generation basis, include the value "SKIP_INTERPRETER" in the field "Constraints" of the configuration, these acronyms are skipped by the SQL generator.

Method	**Description**
generateSql(SqlGeneratorParam sqlGeneratorParameter): SqlGeneratorResult	**Return type:** SqlGeneratorResult **Input**: SqlGeneratorParam sqlGeneratorParameter: parameter structure

Overview of the classes involved:


SqlFilterType
MpdvSqlParam
MpdvSqlParam(String, Object, int)
getParamName() String
getValue() Object
getSqlType() int
toString() String
OperatorType
ASqlFilterExpression
ASqlFilterExpression(SqlFilterType)
getRealExpression() T
getSqlFilterType() SqlFilterType
SqlGeneratorResult
SqlGeneratorResult(List<String>, String, String, String, String, String, List<MpdvSqlParam>)
getAcronymOrderInSelectList() List<String>
getSelectClause() String
getFromClause() String
getWhereClause() String
getGroupByClause() String
getOrderByClause() String
getSqlParameterList() List<MpdvSqlParam>
SqlSpecialFilterResult
SqlSpecialFilterResult(String, List<MpdvSqlParam>)
getSql() String
getSqlParameterList() List<MpdvSqlParam>
SqlFilterSimpleExpression
SqlFilterSimpleExpression(String, OperatorType, List<Object>, DataType)
getRealExpression() T
getKey() String
getOperator() OperatorType
getValues() List<Object>
getValueType() DataType
SpecialParam
SpecialParam(String, OperatorType, Object)
getAcronym() String
getOperator() OperatorType
getValue() Object
SqlFilterRawExpression
SqlFilterRawExpression(String)
getRealExpression() T
getRawClause() String
ISqlSpecialFilter
filter(SqlFilterSimpleExpression) SqlSpecialFilterResult
SqlFilterConjunction
AcronymMergeStruct
AcronymMergeStruct(String, String, ISqlSpecialFilter)
getAcronym() String
getQualifiedDbField() String
getSqlSpecialFilter() ISqlSpecialFilter
SqlFilterComplexExpression
SqlFilterComplexExpression(SqlFilterConjunction, List<ASqlFilterExpression>)
getRealExpression() T
getChildren() List<ASqlFilterExpression>
getConjunction() SqlFilterConjunction
ColumnConfigurator
ColumnConfigurator(boolean, List<String>)
isDistinct() boolean
getRequestedCols() List<String>
SqlGeneratorParam
SqlGeneratorParam(String, List<AcronymMergeStruct>, ColumnConfigurator, List<SpecialParam>, SqlFilterComplexExpression)
getServiceId() String
getAcronymMergeStructList() List<AcronymMergeStruct>
getColumnConfigurator() ColumnConfigurator
getSpecialParamList() List<SpecialParam>
getRootFilterExpression() SqlFilterComplexExpression
ISqlGenerator
generateSql(SqlGeneratorParam) SqlGeneratorResult
Powered by yFiles

1.44.1 Class SqlGeneratorParam

This is the request parameter structure.

Method	Description
getServiceId	Name of the service that is used as basis for generation. **Return type:** String
getAcronymMergeStructList	List with additional or overwriting acronym configurations: All acronyms passed here overwrite the acronyms that might exist in the service configuration. **Return type**: List<AcronymMergeStruct>
getColumnConfigurator	Column configuration to be used: Specifies the number and the order of the selected columns. **Return type**: ColumnConfigurator
getSpecialParamList	List of SpecialParameters to be used **Return type**: List<SpecialParam>
getRootFilterExpression	Root nodes of the FilterParameter in a tree structure. All filter parameters to be used must be included. **Return type:** SqlFilterComplexExpression

1.44.2 Class AcronymMergeStruct

You use this structure to overwrite an acronym configuration in the service used for generation.

Method	Description
getAcronym	Acronym of the service parameter

	Return type: String
getQualifiedDbField	Qualified database field. This means that the structure is as follows: <table alias>.<table field>. You may **not** use %1$s ! **Return type:** String
getSqlSpecialFilter	Callback for a special filtering for the current acronym or NULL if no special filtering is required. A special filtering or user-specific filtering is a filtering that does not have the structure <qualified DB field> <operator> <value>. **Return type:** ISqlSpecialFilter

1.44.3 Interface ISqlSpecialFilter

You use this interface to implement a callback for special filterings or user-defined filterings that do not have the structure <qualified DB field> <operator> <value>.

Method	Description
filter(SqlFilterSimpleExpression simpleExpression) : SqlSpecialFilterResult	Acronym of the service parameter **Return type**: SqlSpecialFilterResult: may not be NULL **Input**: SqlFilterSimpleExpression simpleExpression: parameter structure

1.44.4 Class SqlSpecialFilterResult

This structure includes the SQL snippet for the WHERE condition for the filtering.

Method	Description

getSql()	SQL snippet for WHERE condition. May not be NULL. **Return type:** String
getSqlParameterList()	List of parameters for the SQL snippet. May not be NULL. **Return type**: List<MpdvSqlParam>

1.44.5 Class ASqlFilterExpression

Abstract superclass for all filter parameter structures.

Method	**Description**
getSqlFilterType():SqlFilterType	Type of SQL filter parameter structure **Return type**: SqlFilterType
getRealExpression(): T	Returns the actual type of the filter parameter structure without external CAST. **Return type**: T extends ASqlFilterExpression

1.44.6 Enum SqlFilterType

Includes the type of the SQL filter parameter structure

Value	**Description**
COMPLEX	Node that can have 0 to n child nodes and is either an OR or an AND conjunction.
SIMPLE	Leaf that consists of key, operator and value list.
RAW	Special leaf that includes a blank SQL snippet.

1.44.7 Class SqlFilterComplexExpression

This structure is an AND or an OR conjunction and includes all linked elements as child elements.

Method	Description
getChildren():List<ASqlFilterExpression>	All child elements of this conjunction **Return type**: List<ASqlFilterExpression>
getConjunction():SqlFilterConjunction	Type of conjunction: AND or OR. **Return type**: SqlFilterConjunction

1.44.8 Enum SqlFilterConjunction

Includes the type of the SQL conjunction.

Value	Description
AND	AND conjunction
OR	OR conjunction

1.44.9 Class SqlFilterSimpleExpression

This structure is a filter for an acronym.

Method	Description
getKey(): String	Acronym to be filtered by. **Return type:** String
getOperator(): OperatorType	Operator used for the filtering. **Return type:** OperatorType
getValues(): List<Object>	List of values to be filtered by. **Return type**: List<Object>

getValueType(): DataType	Data type of the values to be filtered by. **Return type**: DataType

1.44.10 Class SqlFilterRawExpression

This structure is a special filter that consists of one WHERE snippet only.

Method	Description
getRawClause(): String	SQL snippet to be directly embedded in the WHERE clause. **Return type:** String

1.45 Interface IMemoryChecker

You can use this utility to register the memory usage in estimated units. A unit can be a cell in a table or 1 kilobyte. If you use this utility, you ensure the stability of Java because the current request is interrupted if the memory demand is too high.

Method	Description
reportAdditionalMemoryConsumption(int units): void	Registers an estimation of the memory demand. This value is added internally to the former estimation. **Input**: int units – the estimated quantity of additionally used "units". **Exception**: If the current request requires too much memory, a RuntimeException is thrown and the request is stopped.

1.46 Interface IMandatorySpecialParameterValidator

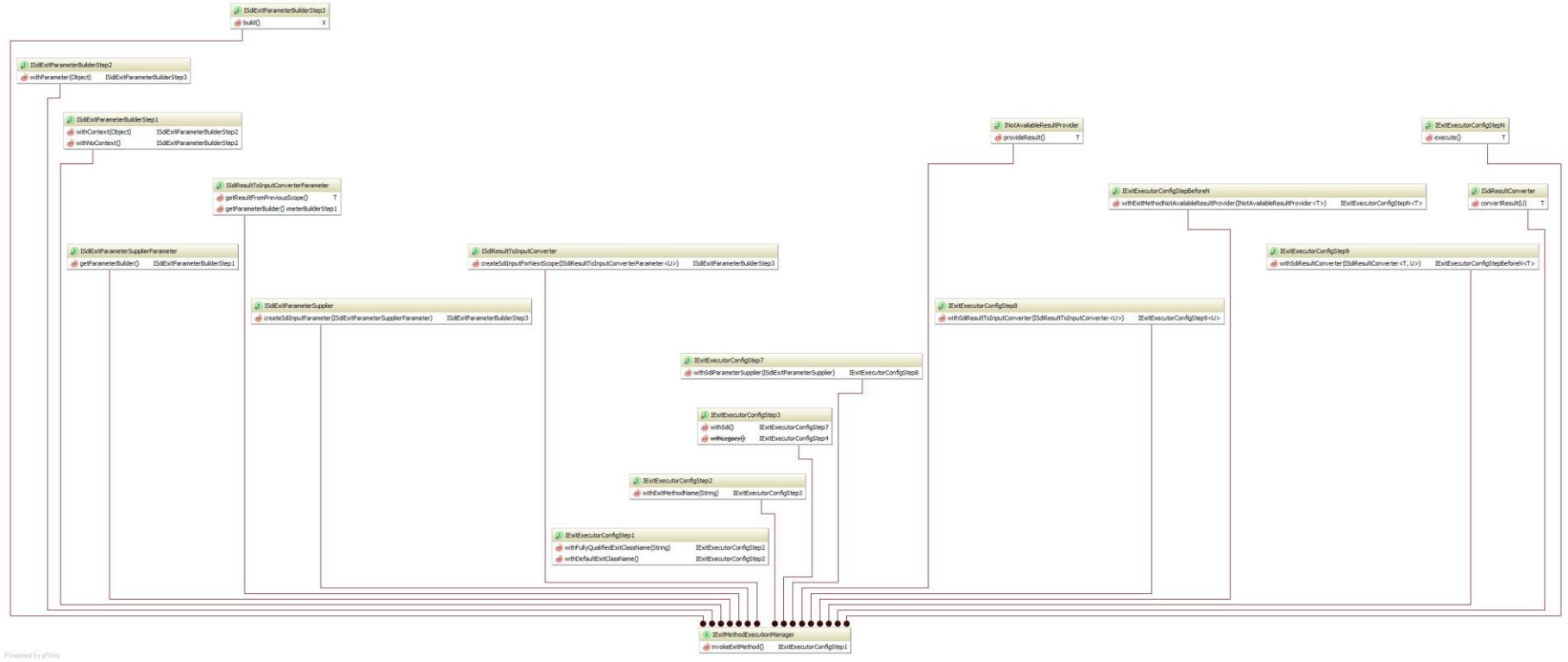
You use this interface to check the configured mandatory parameters of type SpecialParameter in ExternalServices. This interface is interesting for user exits, if other SpecialParameters become mandatory parameters depending on one or several other SpecialParameters.

Method	Description
validateMandatorySpecialParameters(Map<String, SpecialParam> specialParamMap, List<String> additionalMandatoryParameterAcronyms): void	Validates the SpecialParameters using the configuration of the current service. **Input**: specialParamMap – the SpecialParameters to be validated additionalMandatoryParameterAcronyms – a list including optional additional SpecialParameters to be validated that are not configured as mandatory parameters, but are to be treated as such. **Exception**: If one or several mandatory parameters are missing
validateMandatorySpecialParameters(String serviceId, Map<String, SpecialParam> specialParamMap, List<String> additionalMandatoryParameterAcronyms): void	Validates the SpecialParameters using the configuration of the service passed (serviceId parameter). **Input**: serviceId – name of the service whose configuration is to be used for the validation. specialParamMap – the SpecialParameters to be validated

	additionalMandatoryParameterAcronyms – a list including optional additional SpecialParameters to be validated that are not configured as mandatory parameters, but are to be treated as such. **Exception**: If one or several mandatory parameters are missing

1.47 Interface IExitMethodExecutionManager

Method	**Description**
invokeExitMethod: IExitExecutorConfigStep1	Validates the SpecialParameters using the configuration of the current service. **Output**: IExitExecutorConfigStep1: Step Builder that results in the execution of an exit method via the required steps. After each method, only the next possible step is available and the CodeCompletion in the IDEA is supported step by step when an exit method is called.


ISdExitParameterBuilderStep3
build()
ISdExitParameterBuilderStep2
withParameter(Object)
ISdExitParameterBuilderStep3
ISdExitParameterBuilderStep1
withContext(Object)
withNoContext()
ISdExitParameterBuilderStep2
ISdResultToInputConverterParameter
getResultFromPreviousScope()
getParameterBuilder()
ISdExitParameterSupplierParameter
getParameterBuilder()
ISdExitParameterBuilderStep1
ISdResultToInputConverter
createSdiInputForNextScope(ISdResultToInputConverterParameter<U>)
ISdExitParameterBuilderStep3
ISdExitParameterSupplier
createSdiInputParameter(ISdExitParameterSupplierParameter)
ISdExitParameterBuilderStep3
IExitExecutorConfigStep7
withSdiParameterSupplier(ISdExitParameterSupplier)
IExitExecutorConfigStep8
IExitExecutorConfigStep3
withSdi()
withLegacy()
IExitExecutorConfigStep7
IExitExecutorConfigStep4
IExitExecutorConfigStep2
withExitMethodName(String)
IExitExecutorConfigStep3
IExitExecutorConfigStep1
withFullyQualifiedExitClassName(String)
withDefaultExitClassName()
IExitExecutorConfigStep2
IExitExecutorConfigStep2
IExitMethodExecutorManager
invokeExitMethod()
IExitExecutorConfigStep1
INotAvailableResultProvider
provideResult()
T
IExitExecutorConfigStep8
withSdiResultToInputConverter(ISdiResultToInputConverter<U>)
IExitExecutorConfigStep9<U>
IExitExecutorConfigStepBeforeN
withExitMethodNotAvailableResultProvider(INotAvailableResultProvider<T>)
IExitExecutorConfigStepN<T>
IExitExecutorConfigStep9
withSdiResultConverter(ISdiResultConverter<T, U>)
IExitExecutorConfigStepBeforeN<T>
IExitExecutorConfigStepN
execute()
T
ISdiResultConverter
convertResult()
T

```
public void systemUtilFactoryCreatesExitApiWithSdi()
{
    final IExitMethodExecutionManager exitMethodExecutorManager =
this.systemUtilFactory.fetchUtil("ExitMethodExecutionManager");

    final String[] whereConditionHolder = {"b.col6 = 42"};
    final TestResultStructure exitResult = exitMethodExecutorManager.invokeExitMethod()
        .withDefaultExitClassName()
        .withExitMethodName("myExitMethod")
        .withSdi()
        .withSdiParameterSupplier(new ISdiExitParameterSupplier()
        {
            public ISdiExitParameterBuilderStep3 createSdiInputParameter(final
ISdiExitParameterSupplierParameter parameterSupplierParameter)
            {
                return parameterSupplierParameter.getParameterBuilder()
                    .withContext(new InterpretedJavaServiceUeContext(Calendar.getInstance(), "TestUser"))
                    .withParameter(new SdiAugmentSqlParam(
                        "a.col1, b.col2",
                        "testTable a join testTable2 b on a.col3 = b.col4",
                        whereConditionHolder[0],
                        "",
                        "",
                        Collections.<String, SpecialParam>singletonMap(
                            "testService.acronym1", new SpecialParam("testService.acronym1",
OperatorType.EQUAL, Integer.valueOf(42))
                        )
                    ));
            }
        })
        .withSdiResultToInputConverter(new ISdiResultToInputConverter<SdiAugmentSqlResult>()
        {
            public ISdiExitParameterBuilderStep3 createSdiInputForNextScope(final
ISdiResultToInputConverterParameter<SdiAugmentSqlResult> resultToInputConverterParameter)
            {
                final SdiAugmentSqlResult resultFromPreviousScope =
resultToInputConverterParameter.getResultFromPreviousScope();
                // merge the results with the original where condition
                whereConditionHolder[0] = mergeString(whereConditionHolder[0],
resultFromPreviousScope.getWhereSuffix());
                return resultToInputConverterParameter.getParameterBuilder()
                    .withContext(new InterpretedJavaServiceUeContext(Calendar.getInstance(), "TestUser"))
                    .withParameter(new SdiAugmentSqlParam(
                        "a.col1, b.col2",
                        "testTable a join testTable2 b on a.col3 = b.col4",
                        whereConditionHolder[0],
                        "",
                        "",
                        Collections.<String, SpecialParam>singletonMap(
                            "testService.acronym1", new SpecialParam("testService.acronym1",
OperatorType.EQUAL, Integer.valueOf(42))
                        )
                    ));
            }
        }).withSdiResultConverter(new ISdiResultConverter<TestResultStructure, SdiAugmentSqlResult>()
        {
            public TestResultStructure convertResult(final SdiAugmentSqlResult sdiExitResultStructure)
            {
                // merge the results with the previous where condition
                whereConditionHolder[0] = mergeString(whereConditionHolder[0],
sdiExitResultStructure.getWhereSuffix());
                return new TestResultStructure(whereConditionHolder[0]);
            }
        })
        .withExitMethodNotAvailableResultProvider(new INotAvailableResultProvider<TestResultStructure>()
        {
            public TestResultStructure provideResult()
            {
                // no exit was available so use original where condition
                return new TestResultStructure(whereConditionHolder[0]);
            }
        })
        .execute();
    // process exitResult: a
    System.out.println(exitResult.getWhereConditionSuffix());
}
```

```
private String mergeString(final String firstPart,
    final String secondPart)
{
    if(secondPart == null || secondPart.trim().length() == 0)
    {
        return firstPart;
    }
    return firstPart + secondPart;
}

static class TestResultStructure
{
    private final String whereConditionSuffix;

    TestResultStructure(final String whereConditionSuffix)
    {
        this.whereConditionSuffix = whereConditionSuffix;
    }

    public String getWhereConditionSuffix()
    {
        return this.whereConditionSuffix;
    }
}
```

1.48 Interface ISdiDataRowCopyUtil

You can use this util to copy an ISdiDataRow.

Method	**Description**
copyRow(ISdiDataRow „row“): ISdiDataRow	Copies the ISdiDataRow **Input**: ISdiDataRow „row“ – row to be copied **Output**: Copy of the row Note: A direct implementation of ISdiDataRow is not allowed!

1.49 Interface IDataTableToStreamConverter

You can use this util to provide the content of an IDataTable as ISdiDataRowStream.

Method	Description
convert(IDataTable table): ISdiDataRowStream	Provides the content of the IDataTable as stream **Input**: IDataTable table – The table **Output**: Stream that provides the content of the table

1.50 Interface IStreamToDataTableConverter

You can use this util to convert the content, which is provided by a ISdiDataRowStream, into an IDataTable.

Method	Description
convert(ISdiDataRowSteram stream): IDataTable	Iterates all rows that the stream provides and creates an IDataTable. **Input**: ISdiDataRowStream stream – The stream **Output**: A table with the content of the stream.

1.51 Interface IResultTransformationManager

You use this util to apply the result transformations on an ISdiDataRowStream using the repository configuration (via service name). You can also apply own additional transformations (optional).

You can also identify the DB data type for all fields with result transformation.

Method	Description

applyResultTransformations(String serviceName, ISdiDataRowStream stream, List<ISdiResultTransformationCallback> additionalTransformations): ISdiDataRowStream	Applies the result transformations on an ISdiDataRowStream using the repository configuration (via service name). You can also use own additional transformations (optional). **Input**: String serviceName – name of service ISdiDataRowStream stream – The input data stream List<ISdiResultTransformationCallback> additionalTransformations – Own transformation implementations **Output**: A stream with applied transformations
getDbSelectionTypesForTransformedFields(String serviceName): Map<String, DataType>	Some result transformations change the data type of a field. In this case, the repository includes the data type after the transformation. When you read from the DB, you must perhaps use another data type. This method provides the DB type for all fields with result transformation. **Input**: String serviceName – name of service **Output**: A map that includes the DB data type of the acronym (ONLY for fields with result transformations using the repository configuration).

1.52 Auxilliary classes

1.52.1 Class SdiEagerDataRowStream

You can use the SdiEagerDataRowStream class for services to stream rows into the found result set of a service.

Examples on how to use the classes:

„GlobalExits“

Call back function registered with user exit „sdiGlobalAddResultTransformationCallbacks“

„InterpretedJavaService2“

Call back function registered with user exit „sdiAddResultTransformationCallbacks“

The polymorphic constructor of the class allows you to transfer the output rows in various ways:

SdiEagerDataRowStream()

Call without dataRows This variant is used in the above examples of callbacks to delete rows from the result.

SdiEagerDataRowStream(ISdiDataRow dataRow)

Callback with a dataRow. The transferred row is added to the result. This variant is normally used in the above examples of callbacks to modify the row transferred to the callback function and add the modified row to the result.

SdiEagerDataRowStream(List<ISdiDataRow> dataRows)

Callback with a list of dataRows. This list may not be null. The transferred rows are added to the result. This variant is mostly used in the above examples of callbacks to add more rows when individual rows are encountered. Follow the instructions below for creating new rows.

SdiEagerDataRowStream(ISdiDataRow... dataRows)

Callback of several dataRows in form of varialbe parameter list. The transferred rows are added to the result. This variant is mostly used in the above examples of callbacks to add more rows when individual rows are encountered. Follow the instructions below for creating new rows.

If you want to generate instances from ISdiDataRow, always use a factory method!

Do not implement the interface ISdiDataRow yourself!

- For "GlobalExits": Use "getDataRowBuilder()" or "getDataRowPrototypeFactory()" from "SdiGlobalResultTransformationFunctionParameter" to create new rows. You get the currently viewed row with "getDataRow()".

- For "InterpertedJavaService2": Use "ISdiDataRowFactory" from "SdiAddResultTransformationCallbacksParam".